

AI Agent Control Plane

Research & Build Roadmap

A research-first roadmap for turning the original API-dashboard idea into a cost-, continuity-, and resilience-aware system for long-running agent loops.

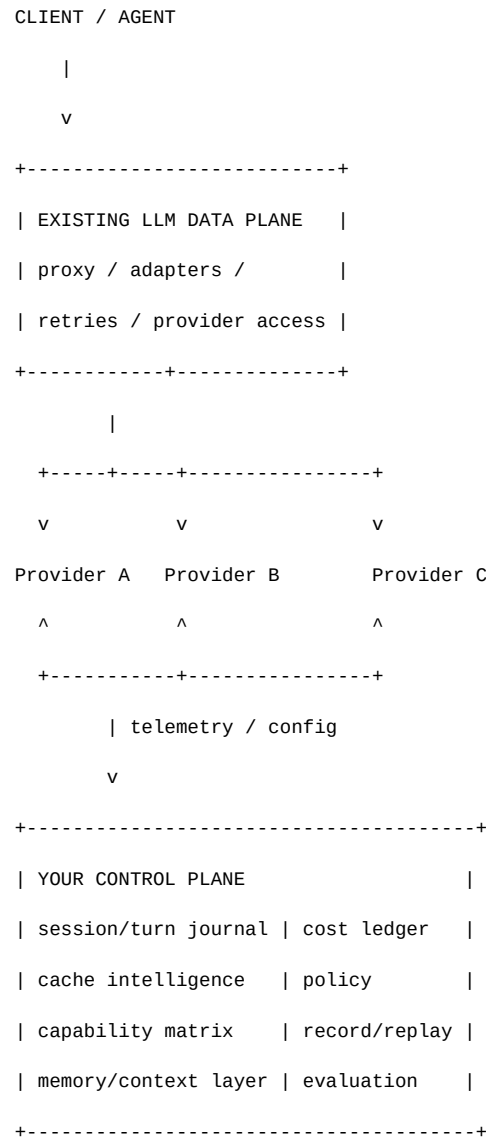
1. Revised problem statement

Original: manage many API keys, track limits, switch APIs, and keep a local context/session alive.

Revised: do not rebuild capabilities already supplied by the LLM data plane. Build the intelligence around it: durable execution, cost optimization, cache-aware routing, safe recovery, policy and evaluation.

Working framing: Cost & Continuity Optimizer for Long-Running AI Agent Loops.

2. Target architecture



3. What the original architecture was missing

Original assumption	Correction
API dashboard is the main product.	Treat it as an interface to a deeper control/optimization layer.
LiteLLM is only a dumb translator.	First audit what the existing data plane already provides; avoid reimplementing routing, fallback, limits and telemetry.

Original assumption	Correction
Session = context window.	Separate durable session state, event history, project memory, active model context, workspace and tool state.
API switching is the hard problem.	The hard cases are cache loss, partial tool execution, idempotency, replay and long-running recovery.
OpenHands needs another session manager.	Current OpenHands already documents ConversationState, EventLog, persistence and resume; identify the actual gap first.
Rotate accounts to bypass limits.	Prefer legitimate provider pools, BYOK/multi-tenant designs and cost-aware multi-provider routing.

4. Research roadmap

Phase 0 — Existing-system reconnaissance

OpenHands Conversation; persistence; event model; pause/resume; context condenser; LLM routing; LiteLLM proxy/router; telemetry. Goal: identify what you should NOT build.

Phase 1 — LLM request lifecycle

Stateless requests; streaming; retries; timeouts; provider errors; tool/function calls; partial responses; provider-specific differences.

Phase 2 — Context engineering

Context window vs memory; history selection; compression; summarization; structured project memory; model-specific limits; cache-aware context construction.

Phase 3 — Agent execution state

Reason → tool call → observation → next step; turn state; task state; checkpoints; workspace/Git state; MCP/tool state; pause/resume; crash recovery.

Phase 4 — Distributed systems

Durability; idempotency; at-least-once effects; retries/backoff; locks/leases; queues; workers; consistency; race conditions; failure detection.

Phase 5 — Workflow orchestration

State machines; DAGs; long-running workflows; checkpoints; human interruption/approval; compensation; replay; workflow versioning.

Phase 6 — Routing intelligence

Provider capability matrix; cost; quota/rate limits; latency; availability; model quality tiers; task-aware degradation; cache affinity; multi-provider routing.

Phase 7 — Correctness boundary

Turn journal; idempotency keys; duplicate side-effect prevention; side-effect classification; transactional boundaries; secrets; audit trail.

Phase 8 — Evaluation

Record/replay; failure injection; router benchmarks; cost/task; recovery success; duplicate tool execution; cache hit rate; latency; task completion.

Phase 9 — Production

Redis only where ephemeral/live coordination is justified; durable database/event store; artifacts; metrics/logs/traces; workers; backpressure; secrets; control-plane failure behavior.

5. The five concepts to understand deeply

Session: durable logical identity of an agent task; independent of provider, key and active context window.

Turn: one logical transition in the agent loop, potentially including an LLM response, tool call and observation.

Journal: append-oriented record of what happened, enabling recovery, auditing and replay.

Context: model-visible information selected for one request. Durable memory can be much larger.

Control plane vs data plane: the data plane serves requests; the control plane owns policy/configuration and consumes telemetry.

6. Real hard problems

- **Prompt-cache affinity:** switching endpoints can discard a warm cache and increase cost/latency.
- **Mid-tool-call failure:** a side-effecting tool may execute while the enclosing model request fails; replay can execute it twice.
- **Cross-model drift:** identical durable state can still produce different plans/tool choices across models.
- **Durable execution:** long-running agent work must survive crashes, network failures and provider failures.
- **Policy correctness:** routing must respect explicit constraints and legitimate provider usage rules.

7. Questions to answer before coding

1. What state does current OpenHands already persist?
2. What does the existing LLM data plane already provide?
3. What quota/cache signals can providers expose?
4. What happens when a tool succeeds but the LLM response is lost?
5. How will tool effects be made replay-safe?
6. What belongs in live state versus durable storage?
7. What happens when the control plane itself fails?
8. How can routing be evaluated without repeatedly spending tokens?

8. First experiment — falsification, not a product build

Do this first: configure a simple agent through the existing data plane with two permitted endpoints. Run a realistic task. Deliberately fail endpoint A. Observe exactly what is preserved, what retries, how tool calls behave, what OpenHands persists, and what telemetry exists. Current OpenHands documentation shows built-in conversation state, event logs, persistence/resume, LLM routing and telemetry, so this experiment is critical before duplicating functionality.

9. Then build the proprietary layer

Recommended order: request ledger → cache-affinity intelligence → turn journal/idempotency boundary → capability matrix → record/replay evaluator → context/memory optimization.

10. Long-term direction

CLIENTS

|

v

OpenAI-compatible / agent-facing interface

|

v

LLM DATA PLANE

|-----> PROVIDERS

| telemetry

v

YOUR CONTROL PLANE

- Policy compiler
- Cost/usage ledger
- Cache intelligence
- Capability matrix
- Session/turn journal
- Context/memory layer
- Record/replay evaluation

11. What to learn first

1. OpenHands Conversation + Persistence + LLM/Routing docs.
2. LiteLLM proxy/router capabilities.
3. Build a tiny LLM agent loop yourself.
4. Add durable turn/event records.
5. Inject failures at every boundary.
6. Learn idempotency and replay using a side-effecting tool.
7. Study context engineering and build a context builder.
8. Add cost/latency/quota telemetry.
9. Prototype cache-aware routing.
10. Build record/replay benchmarks from real traces.

12. Success criteria

Dimension	Measure
Continuity	Correct logical turn is recoverable after infrastructure failure.
Correctness	No duplicate side effects during replay.
Cost	Lower cost per completed task than baseline routing.
Cache efficiency	Improved useful cache-hit ratio / lower switching cost.
Routing	Better decisions under cost, capability and availability constraints.
Resilience	Recoverable after process/provider/network failures.
Portability	Not structurally tied to one agent harness.

13. Bottom line

The original API-dashboard idea is a useful starting point, but the stronger project is the layer that makes long-running agent workloads **recoverable, measurable and cost-efficient**. Do not compete with the existing data plane on generic routing. Research the existing capabilities first, then own the hard boundary: cache-aware optimization, safe tool replay, durable execution semantics, policy-aware decisions and evidence-based evaluation.

Guiding principle: the model/provider endpoint can change; the logical agent work must remain recoverable, measurable and cost-efficient.